

端口复用

STM32 有很多内置外设; 这些外设的外部引脚与 GPIO 复用。

若一个 GPIO 可复用为内置外设的功能引脚, 当该 GPIO 作内置外设使用时, 就称作复用。

端口重映射 8.1.11

每个内置外设都有若干输入输出引脚, 一般每个引脚的输出端口是固定不变的。

但为设计工程师可更好地安排引脚的走向和功能, 引入了外设引脚重映射。

一个外设的引脚除了有默认端口, 也可通过设置重映射寄存器, 把该外设引脚映射到其它端口。

例如, PA9、PA10 可复用为串口发送、接收引脚, 但有时需其它引脚来实现, PB6、PB7 就可完成该任务

如何开启时钟?

- ① GPIO 功能时钟直接开启。
- ② 复用功能时钟 (+ GPIO 功能时钟)
- ③ AFIO 辅助功能时钟

何时开启 AFIO 时钟?

AFIO_MAPR: 配置复用功能重映射

AFIO_EXTICRx: 配置外部中断线映射

AFIO-EVCR: 配置 EVENTOUT 事件输出

对寄存器 AFIO_MAPR、AFIO_EXTICRx、AFIO-EVCR 进行读写操作前, 应首先打开 AFIO 时钟

param1: 传给所在总线下的外设

param2: 当前使用状态

```
void RCC - APB1PeriphClockCmd( uint32_t RCC-APB1Periph, FunctionalState NewState)
```

```
void RCC - APB2PeriphClockCmd( uint32_t RCC-APB2Periph, FunctionalState NewState)
```

流水灯

共阳极接法: 将8位流水灯LED1~LED8的阴极与单片机的I/O端口相连,
当单片机端口相应的位输出低电平时对应LED灯被点亮,反之熄灭

设PA口用作输出口, PA0控制LED1 ... PA7控制LED8

```
int main (void)
```

```
{  
    MyDelay (0x7fffff); // 软件延时  
    LED_Init(); // PA0~PA7分别对应控制LED1~LED8, PA口初始化为推挽式输出  
    while(1)  
    {  
        GPIO_ResetBits (GPIOA, GPIO_Pin_0);  
        GPIO_SetBits (GPIOA, GPIO_Pin_1);  
        ⋮  
        GPIO_SetBits (GPIOA, GPIO_Pin_7);  
        MyDelay (0x7fffff);  
        GPIO_SetBits (GPIOA, GPIO_Pin_0);  
        GPIO_ResetBits (GPIOA, GPIO_Pin_1);  
        ⋮  
        GPIO_ResetBits (GPIOA, GPIO_Pin_7);  
        MyDelay (0x7fffff);  
    }  
}
```

```
void LED_Init(void)
```

```
{  
    GPIO_InitTypeDef GPIO_InitStructure;  
    RCC_APB2PeriphClockCmd (RCC_APB2Periph_GPIOA, ENABLE); // 开启PA口对应时钟  
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2 ... | GPIO_Pin_7;  
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; // 推挽输出模式  
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; // 输出频率 50MHz  
    GPIO_Init (GPIOA, &GPIO_InitStructure); // 根据制定参数初始化PA口  
}
```

8段数码管

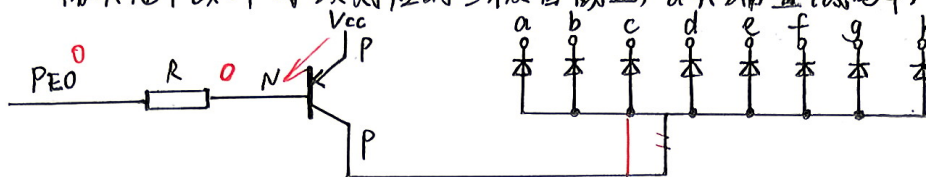
设 $P00 \sim P07$ 对应 $a \sim h$, 控制字形; 设 $P10 \sim P17$ 对应从左往右的8个位, 控制字位。

设 $PA7$ 接 $P07, \dots, PA0$ 接 $P00$, 对共阳极接法, 要想显示 "1" 需对 PA 口赋值 $0xF9$ 。

设 $PE0$ 接 $P10, \dots, PE7$ 接 $P17$, 要在左边第一个数码管显示, 要令 PE 口输出 $0xFE$ 。

当 $PE0=0$ 时, PNP 三极管导通, 对应的数码管公共端置高电平, 可显示字形,

而其他 $PEx=1$ 导致对应的三极管截止, 公共端置低电平, 不显示字形



扫描显示, 一次激活一个
利用视觉暂留

void LED-Init (void).

```
{  
  GPIO_InitTypeDef  GPIO_InitStructure;
```

```
  RCC->APB2PeriphClockCmd(RCC->APB2Periph-GPIOA, ENABLE); //使能PA口时钟
```

```
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin-0 | ... | GPIO_Pin-7;
```

```
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode-Out-PP; //推挽输出
```

```
  GPIO_InitStructure.GPIO_Speed = GPIO_Speed-50MHz;
```

```
  GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
  GPIO_SetBits(GPIOA, GPIO_Pin-0 | ... | GPIO_Pin-7); //初始全部熄灭
```

```
  RCC->APB2PeriphClockCmd(RCC->APB2Periph-GPIOE, ENABLE);
```

```
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin-0 | ... | GPIO_Pin-7;
```

```
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode-Out-PP;
```

```
  GPIO_Init(GPIOE, &GPIO_InitStructure);
```

```
  GPIO_SetBits(GPIOE, GPIO_Pin-0 | ... | GPIO_Pin-7); //初始全部熄灭
```

```
}
```

$PA0 \sim PA7$:

```
#define DX0 PAout(0)
```

```
...
```

```
#define DX7 PAout(7)
```

```
u8 seg-tab[10] = {0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90};
```

```
void Display(u8 index)
```

```
{  
  PA0DX0 = seg-tab[index] & 0x01;
```

```
  DX1 = (seg-tab[index] >> 1) & 0x01;
```

```
  DX2 = (seg-tab[index] >> 2) & 0x01;
```

```
  ...
```

```
  DX7 = (seg-tab[index] >> 7) & 0x01; //第1个数码管显示数字0
```

```
}
```

每次右移1位, 取最低位赋值给对应引脚;
//把seg-tab[index]的每一位赋值给对应8
引脚, 实现该位中的段选控制

$PE0 \sim PE7$:

```
#define D0 PEout(0)
```

```
...
```

```
#define D7 PEout(7)
```

```
int main(void)
```

```
{  
  delay_init(); //延时初始化函数
```

```
  LED_Init(); //数码管初始化函数
```

```
  while(1)
```

```
  {  
    Display(0); D0=0; delay-ms(1); D0=1;
```

```
    Display(1); D1=0; delay-ms(1); D1=1;
```

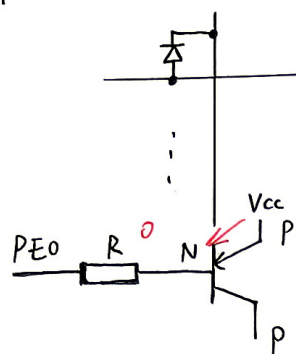
```
    ...  
    Display(7); D7=0; delay-ms(1); D7=1;
```

```
  }
```

```
}
```


8x8 点阵模块

P07~P00 (上~下) 处理行值, P27~P20 (左~右) 处理列值



P07 输出 1, P27 输出 0, 左上角第一个点亮

1 0
0 x

当 PEO 置为低电平时, 三极管导通, 点阵第一行的公共端置高电平, 要点亮第一行某 LED, 将其对应 PAX 置为低电平即可

// 将 PA 口低 8 位和 PEO 低 8 位设为推挽输出模式

Void LED-Init (Void)

{

GPIO-InitTypeDef GPIO_InitStructure;

RCC-APB2PeriphClockCmd (RCC-APB2Periph-GPIOA, ENABLE); // 使能 PA 口时钟

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | ... | GPIO_Pin_7;

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;

GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

GPIO_Init (GPIOA, &GPIO_InitStructure);

GPIO_SetBits (GPIOA, GPIO_Pin_0 | ... | GPIO_Pin_7);

RCC-APB2PeriphClockCmd (RCC-APB2Periph-GPIOE, ENABLE); // 使能 PE 口时钟

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | ... | GPIO_Pin_7;

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;

GPIO_Init (GPIOE, &GPIO_InitStructure);

GPIO_SetBits (GPIOE, GPIO_Pin_0 | ... | GPIO_Pin_7);

}

// PA7~PA0 与 P27~P20 对应连接, 根据起始索引值查表处理某 1 行的 8 列

us seg-tab[88] = { 0xff, 0x99, 0x66, 0x7e, 0x7e, 0xbd, 0xdb, 0xe7, // 0

0xe7, 0xdb, 0xdb, 0xdb, 0xdb, 0xdb, 0xdb, 0xe7, // 0

⋮

0xc3, 0x99, 0x99, 0x81, 0xc1, 0xf9, 0xf9, 0xc3 // 9 }

#define DX0 PAout(0)

⋮

#define DX7 PAout(7)

void Display(us index) // 从字模数组中取出数据, 处理某行的 8 列.

{

DX0 = seg-tab[index]&0x01;

DX1 = (seg-tab[index]>>1)&0x01;

DX2 = (seg-tab[index]>>2)&0x01;

⋮

DX7 = (seg-tab[index]>>7)&0x01;

}

// PE7~PE0 与 P07~P00 对应连接, 对应位输出0, 则整行擦除; 对应位输出1, 则根据列数据显示对应行
(连续这样处理8行就能使8*8点阵字形呈现)

```
#define D0 PEout(0)
```

```
;
```

```
#define D7 PEout(7)
```

```
void zero(void) // 字模表中索引值为8~15的位置存放数字0的8*8的点阵数据
```

```
{
```

```
    Display(8);
```

```
    D0 = 0; // 选通该行, 擦除旧数据并准备显示新内容
```

```
    delay_ms(1);
```

```
    D0 = 1; // 将行选通信号拉高以锁定该行显示内容, 保证稳定性和准确性, 避免残影/闪烁
```

```
    ;
```

```
    Display(15);
```

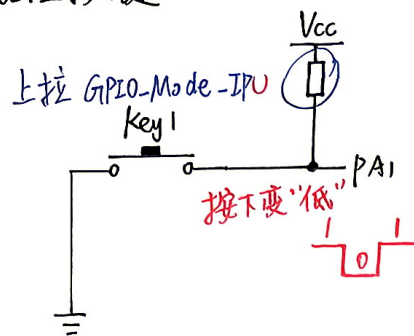
```
    D7 = 0;
```

```
    delay_ms(1);
```

```
    D7 = 1;
```

```
}
```

独立按键



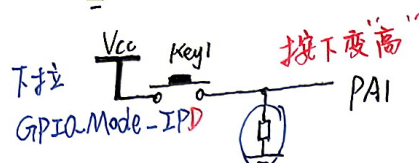
每个按键与单片机的一位I/O相连, 每个按键按下与否不对其他按键连接的单片机I/O产生影响。

当按键Key1一端接地, 另一端接PA1

当按键未按下, 若PA1内部有上拉电阻, 则其输入为高电平

当按键被按下后, PA1变为低电平。

只需在程序中判断引脚PA1的状态, 就可以知道Key1是否处于闭合状态



设PE1~PE4对应检测Key1~Key4 输入 按键的状态, PC1~PC4对应控制LED1~LED4 输出 是否点亮

每次按下Keyx, LEDx状态翻转

GPIO模式常量: GPIO_Mode_TypeDef

GPIO_Mode_AIN

模拟输入模式 // 一般用于ADC采集模拟电压

GPIO_Mode_IN_FLOATING

浮空输入模式 (默认) // 输入阻抗较大, 一般用于I2C、USART通信协议接收端

GPIO_Mode_IPU

上拉输入模式

GPIO_Mode_IPD

下拉输入模式

GPIO_Mode_Out_PP

通用推挽输出模式

GPIO_Mode_Out_OD

通用开漏输出模式

GPIO_Mode_AF_ODRPP

复用推挽输出模式

GPIO_Mode_AF_ODRPP

复用开漏输出模式

对独立按键查询而言, 若有外部上拉电阻, 引脚可配置为浮空输入模式;

若无外部上拉电阻, 引脚可配置为上拉输入模式, 实现内部上拉。

相关库函数: GPIO_Init // 依据GPIO_InitStruct中指定的参数初始化外设GPIOx寄存器

GPIO_ReadInputDataBit // 读取指定端口管脚输入

GPIO_SetBits // 设置指定的数据端口位

GPIO_ResetBits // 清除指定的数据端口位

GPIO_WriteBit // 设置或清除指定的数据端口位

读入GPIO引脚值:

GPIO_ReadInputData // 读取整个端口的16位

GPIO_ReadInputDataBit // 读取某个位 ← 按键检测用这个更方便!

uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)

↓
返回值: 输入引脚的数值, 0或1

指定端口

GPIOA~GPIOG

指定引脚

GPIO_Pin_0~GPIO_Pin_15

```
int main(void)
```

```
{
```

```
LED-GPIO-Config(); // 配置LED对应端口
```

```
Key-GPIO-Config(); // 配置按键对应端口
```

```
while(1)
```

LED 状态翻转

```
{
```

```
if (Key-Scan(GPIOE, GPIO-Pin-1) == KEY_ON) // 扫描按键, 有一次按键动作就让LED翻转
```

LED1
反转
:

```
{ GPIO_WriteBit(GPIOC, GPIO-Pin-1,
```

```
(BitAction)(1-(GPIO_ReadOutputDataBit(GPIOC, GPIO-Pin-1))));
```

强制类型转换

```
}
```

```
:
```

LED4
反转
}

```
if (Key-Scan(GPIOE, GPIO-Pin-4) == KEY_ON)
```

```
{ GPIO_WriteBit(GPIOC, GPIO-Pin-4,
```

```
(BitAction)(1-(GPIO_ReadOutputDataBit(GPIOC, GPIO-Pin-4))));
```

```
}
```

```
}
```

```
void LED-GPIO-Config(void)
```

```
{
```

```
GPIO_InitTypeDef GPIO_InitStructure;
```

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE); // 激活GPIOC对应时钟
```

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin-1 | GPIO_Pin-2 | GPIO_Pin-3 | GPIO_Pin-4; // 使用引脚PC1~PC4
```

```
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; // 推挽输出
```

```
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; // 输出频率 50MHz
```

```
GPIO_Init(GPIOC, &GPIO_InitStructure); // 按上述初始化GPIOC
```

```
GPIO_SetBits(GPIOC, GPIO_Pin-1 | GPIO_Pin-2 | GPIO_Pin-3 | GPIO_Pin-4); // 熄灭LED1~LED4
```

```
void Key-GPIO-Config(void)
```

```
{
```

```
GPIO_InitTypeDef GPIO_InitStructure;
```

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);
```

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin-1 | GPIO_Pin-2 | GPIO_Pin-3 | GPIO_Pin-4;
```

```
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
```

```
GPIO_Init(GPIOE, &GPIO_InitStructure);
```

```
}
```


// 按键扫描

```
#define KEY_ON 0
#define KEY_OFF 1

uint8_t Key_Scan(GPIO_TypeDef * GPIOx, uint GPIO_Pin)
{
    // 检测是否有按键被按下
    if (GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON) ↓ 0 不
    {
        Delay(10000); // 延时消抖
        if (GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON)
        {
            while (GPIO_ReadInputDataBit(GPIOx, GPIO_Pin) == KEY_ON); // 等待按键释放
            return KEY_ON;
        }
    }
    else
        return KEY_OFF;
}
else
    return KEY_OFF;
}
```

拓展: LED 状态翻转的位带写法

```
#define BITBAND(addr, bitnum) \
    ((addr & 0xF0000000) + 0x2000000 ((addr & 0xFFFFF) << 5) + (bitnum << 2))

#define MEM_ADDR(addr)    *((volatile unsigned long *) (addr))

#define BIT_ADDR(addr, bitnum)    MEM_ADDR(BITBAND(addr, bitnum))

#define GPIOC_IDR_Addr    (GPIOC_BASE + 8)    // 0x40011008
#define GPIOC_ODR_Addr    (GPIOC_BASE + 12)    // 0x4001100c

#define PCout(n)    BIT_ADDR(GPIOC_ODR_Addr, n)    // 输出
#define PCin(n)    BIT_ADDR(GPIOC_IDR_Addr, n)    // 输入

#define OldValue    PCin(3)
#define NewValue    PCout(3)

NewValue = ~ OldValue;
```

矩阵键盘

行扫描法:

先判断有无按键按下: 将全部列线 $L1 \sim L4$ 置高电平, 检测行线 $H1 \sim H4$ 的状态.

只要有一行的电平是高电平, 就表明有按键被按下且位于高电平行线与4根列线交叉的4个按键中
如果没有, 就说明无按键按下.

再判断闭合键所在位置: 依次将列线置为高电平, 读出行线值, 根据不同的行线值即可区分位于当前列不同行的键

(例如, 若 $L1=1$ 时读出 $H1H2H3H4 = 1000$, 则表明第1行第1列按键被按下)
($L2=L3=L4=0$)

行列反转法:

先设定列线为输出, 行线为输入.

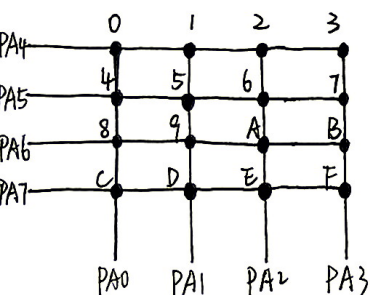
将所有列线置低电平, 通过行扫描读取行值. 如果有行值为0, 则说明有按键按下且位于该行.

再设定行线为输出, 列线为输入.

将所有行线置低电平, 通过列扫描读取列值, 列值为0的列就是按键所在列.

行列交叉点处的键就是闭合键

例. 矩阵键盘读取, 在数码管上显示键号 ($0 \sim 9, A \sim F$)



主程序

```
int main()
{
    s1b kval;

    SysTick-Init(); // 初始化系统时钟
    Keyboard-Init(); // 初始化矩阵键盘
    NixieTubes-Init(); // 用GPIOA、GPIOB 控制数码管

    while(1)
    {
        kval = Read-KeyVal();
        NixieDisplay(kval);
    }
}
```

// Keyboard - Init: 按键端口初始化

```
#define COL_ALL GPIO-Pin-0 | GPIO-Pin-1 | GPIO-Pin-2 | GPIO-Pin-3
```

```
#define ROW_ALL GPIO-Pin-4 | GPIO-Pin-5 | GPIO-Pin-6 | GPIO-Pin-7
```

```
void Keyboard-Init (void)
```

```
{
```

```
    GPIO_InitTypeDef GPIO_InitStructure;
```

```
    RCC-APB2PeriphClockCmd (RCC-APB2Periph-GPIOA, ENABLE);
```

```
    GPIO_InitStructure.GPIO-Pin = COL-ALL; // 列
```

```
    GPIO_InitStructure.GPIO-Speed = GPIO-Speed-10MHz;
```

```
    GPIO_InitStructure.GPIO-Mode = GPIO-Mode-Out-PP; // 推挽输出
```

```
    GPIO_Init (GPIOA, &GPIO_InitStructure);
```

```
    GPIO_InitStructure.GPIO-Pin = ROW-ALL; // 行
```

```
    GPIO_InitStructure.GPIO-Speed = GPIO-Speed-10MHz;
```

```
    GPIO_InitStructure.GPIO-Mode = GPIO-Mode-IPD; // 下拉输入
```

```
    GPIO_Init (GPIOA, &GPIO_InitStructure); // 默认低电平, 要是被按下, 电平就会被拉高)
```

```
    GPIO_SetBits (GPIOA, COL-ALL); // 列线全部置为高电平
```

```
}
```

// NixieTube - Init 数码管接口初始化

```
void NixieTubes-Init (void)
```

```
{
```

```
    GPIO_InitTypeDef GPIO_InitStructure;
```

```
    RCC-APB2PeriphClockCmd (RCC-APB2Periph-GPIOD | RCC-APB2Periph-GPIOE, ENABLE);
```

```
    GPIO_InitStructure.GPIO-Pin = 0xff; // 使用 Pin0 ~ Pin7
```

```
    GPIO_InitStructure.GPIO-Mode = GPIO-Mode-Out-PP;
```

```
    GPIO_InitStructure.GPIO-Speed = GPIO-Speed-50MHz;
```

```
    GPIO_Init (GPIOD, &GPIO_InitStructure); // GPIOD 作位选
```

```
    GPIO_Init (GPIOE, &GPIO_InitStructure); // GPIOE 作段选
```

```
}
```


// Keyscan 按键扫描

static Read-KeyValue(void)

{

static KeyValue = -1; // 无按键

if((GPIO_ReadInputData(GPIOA) & 0xff) != 0x0f) // 列线全置1后读入的行线值不等于全0,表明
(可能)有键按下

{

Delay_ms(2); // 软件消抖

if((GPIO_ReadInputData(GPIOA) & 0xff) != 0x0f) // 延时等待后若读入行值仍不全为0,则
确认有键按下

{

GPIO_SetBits(GPIOA, GPIO_Pin_0); // 仅第1列列线置1,检查按键是否位于第1列

GPIO_ResetBits(GPIOA, GPIO_Pin_1 | GPIO_Pin_2 | GPIO_Pin_3); // 第2,3,4列列线置0

switch(GPIO_ReadInputData(GPIOA) & 0xff) // 获取PA口低8位 PA[7...4] PA[3...0]

{

case 0x11: KeyValue = 0; break; // PA[7...4] PA[3...0] = 0001-0001, 表明第1行第1列键按下

case 0x21: KeyValue = 4; break; // 0010-0001 2 1

case 0x41: KeyValue = 8; break; // 0100-0001 3 1

case 0x81: KeyValue = 0x0C; break; // 1000-0001 4 1

}

;

GPIO_SetBits(GPIOA, GPIO_Pin_3); // 仅第4列列线置1,检查按键是否位于第4列

GPIO_ResetBits(GPIOA, GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2);

switch(GPIO_ReadInputData(GPIOA) & 0xff)

{

case 0x18: KeyValue = 3; break;

case 0x28: KeyValue = 7; break;

case 0x48: KeyValue = 0x0B; break;

case 0x88: KeyValue = 0x0F; break;

}

GPIO_SetBits(GPIOA, COL_ALL); // 全部列线置1 按下: 0 1 0 (1行)

while((GPIO_ReadInputData(GPIOA) & 0xff) != 0x0f); // 等待按键释放

return KeyValue;

}

return -1; // 无键按下

}

return -1; // 无键按下

}

// NixieDisplay 数码管显示

-1 不显示

void NixieDisplay (s1b value) 0~9, ANF (>0 显示)

{

// (共阴接法) ←

DP G F E D C B A 1] 字形码

0 0 1 1 1 1 1 1

uint8_t num[16] = { 0x3f, 0x06, 0x5b, 0x4f,
0x66, 0x6d, 0x7d, 0x07,
0x7f, 0x6f, 0x77, 0x7c,
0x39, 0x5e, 0x79, 0x71 };

s1b j;

j = value;

if (j >= 0)

{ // PD[7...0] 对应选择从左到右 8 个数码管, 低电平选用, 高电平禁用
GPIOC → ODR = 0xff; // 关闭全部显示

GPIOE → ODR = ~num[j]; // 字形表按共阴接法设计, 共阳接法取反使用即可.

GPIOC → ODR = 0x00; // 选择全部管子

Delay_ms(2);

}

}

中断原理回顾

CPU和外设数据传送方式 { 程序查询传送
中断传送
DMA 传送

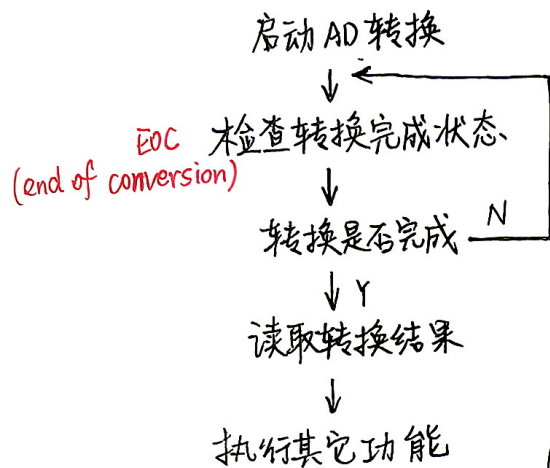
需要中断的原因: 便于处理异步事件, 提高CPU利用率

中断到来以后, CPU需要: 保存断点, 转向中断服务程序

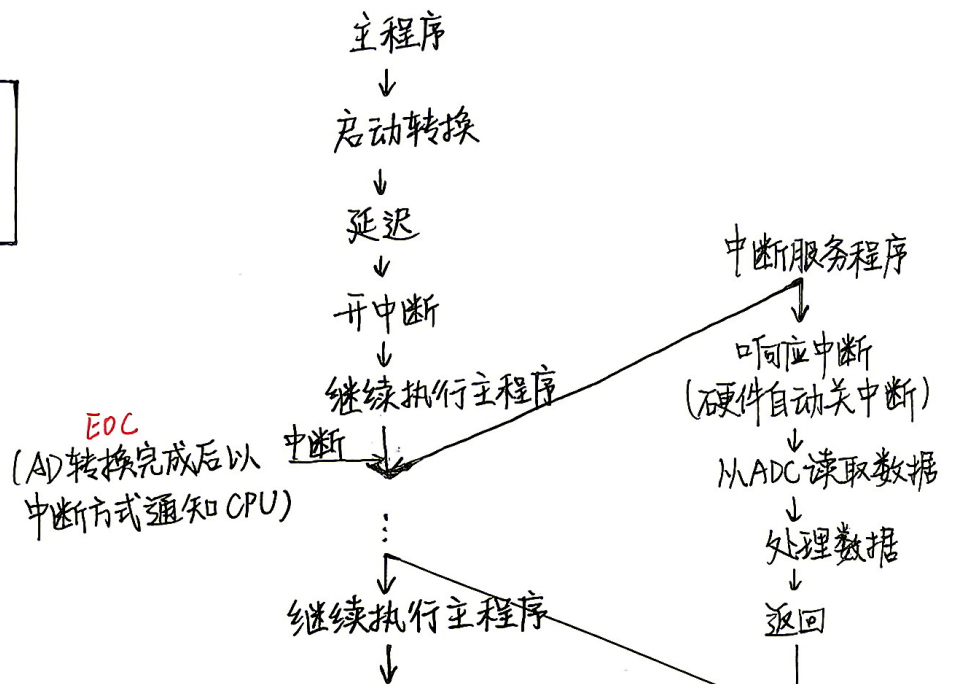
中断处理完成以后: 返回断点处继续执行被中断打断的程序

以数据采集中 A/D 转换任务为例:

查询方式读取AD转换结果



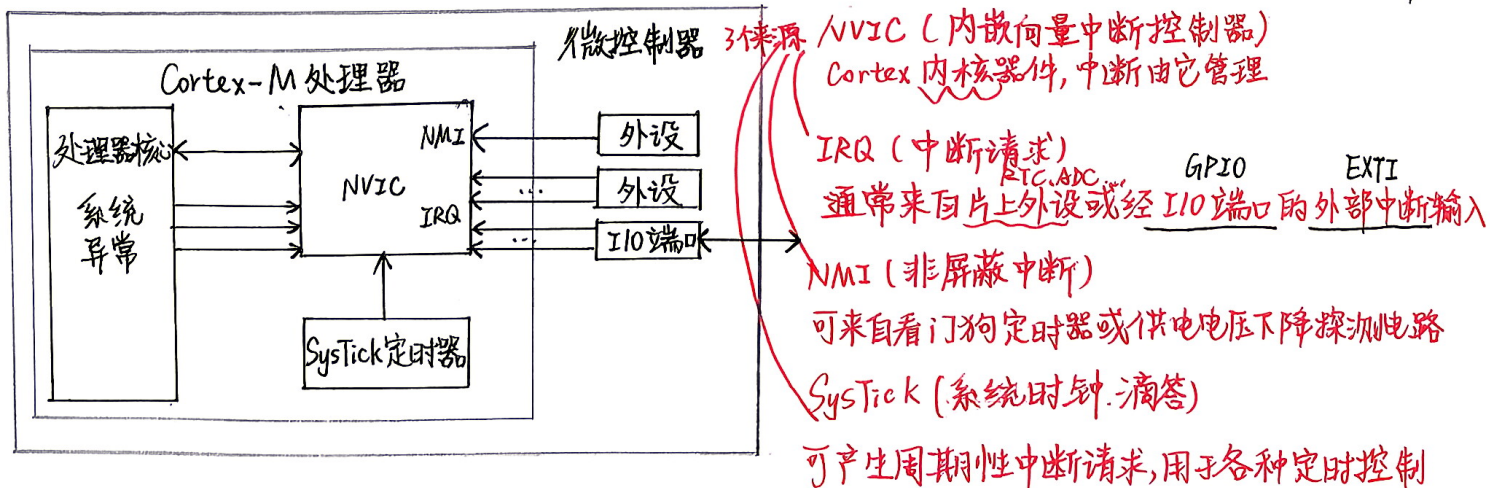
中断方式读取AD转换结果



Cortex内核具有强大的异常响应系统, 将能打断当前代码执行流程的事件分为异常和中断, 编号0~15的是内核异常, 16以上的称为外部中断, 这个表就是中断向量表 exception interrupt

STM32重新排表, 将编号-3~6的中断向量定义为系统异常, 从7开始的为外部中断

(编号为负的内核异常不能被设置优先级, 如复位Reset、不可屏蔽中断NMI、硬错误Hardfault)



抢占优先级和响应优先级

STM32的中断向量有2个属性：抢占属性、响应属性

属性编号越小表明优先级别越高

抢占：打断其他中断的属性。因为这个属性就产生了嵌套中断

(在执行中断服务函数A的过程中被中断B打断,执行完中断服务函数B再继续执行中断服务函数A)

响应：当抢占属性相同时,若两个中断同时到达,优先处理响应优先级高的中断。

例如. 现在有3个中断向量

中断向量	抢占优先级	响应优先级
A	0	0
B	1	0
C	1	1

若内核正在执行C的中断服务函数,则它不能被抢占优先级更高的中断A打断(由于B、C抢占优先级相同,故C不能被B打断)

若B、C中断同时到达,则内核首先响应响应优先级更高的B中断

NVIC优先级配置

STM32使用4位优先级寄存器,支持16种中断向量的优先级

4位分成抢占组优先级、响应(子)优先级,有5种分配形式

STM32的5组优先级配置

组号	组合方式	抢占优先级	响应优先级
0	0-4	无	4位配置响应优先级 16种响应优先级(0~15)
1	1-3	1位配置抢占优先级 2种抢占优先级(0、1)	3位配置响应优先级 8种响应优先级(0~7)
2	2-2	2位配置抢占优先级 4种抢占优先级(0~3)	2位配置响应优先级 4种响应优先级(0~3)
3	3-1	3位配置抢占优先级 8种抢占优先级(0~7)	1位配置响应优先级 2种响应优先级(0、1)
4	4-0	4位配置抢占优先级 16种抢占优先级(0~15)	无

EXTI (External Interrupt) 外部中断

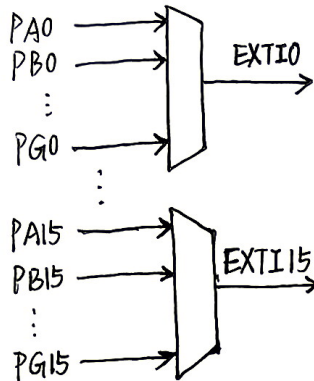
- STM32 所有 I/O 端口都可以配置为 EXTI 外部中断模式, 用来捕捉外部信号。
用来检测外设中断请求, 可配置为下降沿触发、上升沿触发、上升下降沿触发
- 所有 GPIO 引脚都可以作为中断源 (IO 功能强大的表现之一)
- 众多 GPIO 引脚通过 MUX 连接到 16 个 EXTI 外部中断线上

共用中断向量的中断通道不能嵌套!

$PA_x \sim PG_x$ 引脚连接到 $EXTI_x$, 同一时刻 $EXTI_x$ 只能响应一个端口引脚 x 的中断事件
分时复用!

$EXTI_9 \sim EXTI_5$ 共用一个中断向量, 称为 $EXTI_{9-5}$

$EXTI_{15} \sim EXTI_{10}$ 共用一个中断向量, 称为 $EXTI_{15-10}$



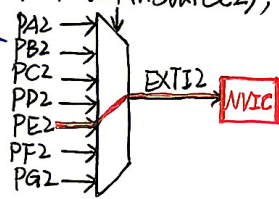
配置 GPIO 引脚与中断线的映射关系靠函数 $GPIO_EXTI_LineConfig()$ 实现:

```
void GPIO_EXTI_LineConfig(uint8_t GPIO_PortSource,
                          uint8_t GPIO_PinSource);
```

例如:

```
GPIO_EXTI_LineConfig(GPIO_PortSourceGPIOE, GPIO_PinSource2);
// 将 GPIOE.2 与 EXTI2 中断线连接起来
```

实现 GPIO 的 PE2 引脚与
EXTI2 中断线的映射关系



STM32 库在 $stm32f10x_exti.c$ 中提供了 8 个 EXTI 函数

$EXTI_Init$	// EXTI 初始化
$EXTI_DeInit$	// EXTI 解除初始化
$EXTI_StructInit$	// 填充 EXTI 初始化结构成员
<u>$EXTI_GetITStatus$</u>	<u>// 获取中断状态</u>
<u>$EXTI_ClearITPendingBit$</u>	<u>// 中断挂起位清除</u>
$EXTI_GetFlagStatus$	// 获取标志状态
$EXTI_ClearFlag$	// 挂起标志清除
$EXTI_GenerateSWInterrupt$	// 产生软件中断

EXTI 初始化:

定义: void EXTI_Init(EXTI_InitTypeDef * EXTI_InitStructure);

```
typedef struct
{
    uint32_t EXTI_Line; // 中断线标号(取值范围 EXTI_Line0 ~ EXTI_Line15)
    EXTI_Mode_TypeDef EXTI_Mode; // 中断模式(可选值: EXTI_Mode-Interrupt, EXTI_Mode-Event)
    EXTI_Trigger_TypeDef EXTI_Trigger; // 触发方式(可选值: EXTI_Trigger-Falling, EXTI_Trigger-Rising, EXTI_Trigger-Rising-Falling)
    FunctionalState EXTI_LineCmd; // 允许 or 禁止
} EXTI_InitTypeDef;
```

例如:

```
EXTI_InitTypeDef EXTI_InitStructure;
EXTI_InitStructure.EXTI_Line = EXTI_Line4;
EXTI_InitStructure.EXTI_Mode = EXTI_Mode-Interrupt;
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger-Falling;
EXTI_InitStructure.EXTI_LineCmd = ENABLE;
EXTI_Init(&EXTI_InitStructure); // 依据 EXTI_InitStructure 中指定的参数初始化外设 EXTI 寄存器
```

在写中断服务函数时常使用的两个函数:

```
ITStatus EXTI_GetITStatus(uint32_t EXTI_Line);
// 一般用在开头, 判断某中断线上的中断是否发生(标志位是否置位)
void EXTI_ClearITPendingBit(uint32_t EXTI_Line);
// 一般用在结束前, 清除某中断线上的中断标志位以防止中断阻塞
```

例如:

```
void EXTI3_IRQHandler(void)
{
    if (EXTI_GetITStatus(EXTI_Line3) != RESET)
    {
        // (中断逻辑)
    }
    EXTI_ClearITPendingBit(EXTI_Line3);
}
```

EOL. 否则会阻止同一根线上的中断
在它返回之前
再次被响应

另外:

```
FlagStatus EXTI_GetFlagStatus(EXTI_Line8);
// 判断外部中断状态

EXTI_ClearFlag(EXTI_Line2);
// 清除外部状态标志位
```

EXTI_GetITStatus 先判断该中断是否使能, 使能了才去判断中断标志位.

EXTI_GetFlagStatus 直接判断状态标志位

使用 NVIC 之前一定要配置 NVIC, STM32 库在 misc.c 中提供了 NVIC 配置函数, 使用中断时一定要在项目中添加 misc.c 文件

NVIC_PriorityGroupConfig	配置优先级组合方式
NVIC_Init	NVIC 初始化
NVIC_SetVectorTable	设置向量表位置和偏移
NVIC_SystemLPConfig	为系统进入低功耗模式选择条件
SysTick_CLKSourceConfig	配置系统时钟 SysTick 时钟源

NVIC 优先级分组函数

函数声明: void NVIC_PriorityGroupConfig (uint32_t NVIC_PriorityGroup);

这个函数在系统中只能被调用一次, 一旦分组确定最好不要修改

NVIC_PriorityGroupConfig (NVIC_PriorityGroup-0);

0-4 组合

1	1-3
2	2-2
3	3-1
4	4-0

NVIC 初始化函数

函数声明: void NVIC_Init (NVIC_InitTypeDef * NVIC_InitStruct)

结构体成员

结构体

typedef struct

{
uint8_t NVIC_IRQChannel; // 定义初始化的是哪个中断, 可在 stm32f10x.h 中找到
// 每个中断对应的名字 (如 USART1_IRQn)
uint8_t NVIC_IRQChannelPreemptionPriority; // 抢占优先级
uint8_t NVIC_IRQChannelSubPriority; // 响应优先级
FunctionalState NVIC_IRQChannelCmd; // 中断通道使能 or 失能
} NVIC_InitTypeDef;

举例: 要使能串口的中断, 同时设置抢占优先级为 1, 响应优先级为 2, 初始化写法:

NVIC_InitTypeDef NVIC_InitStructure;

NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn; // 串口中断

NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1; // 抢占优先级为 1

NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2; // 响应优先级为 1

NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; // IRQ 通道使能

NVIC_Init(&NVIC_InitStructure); // 依据上面指定的参数初始化 NVIC 寄存器

中断优先级设置步骤:

① 设置中断分组, 确定组号 (即确定抢占优先级和响应优先级分配位数), 调用函数为 NVIC_PriorityGroupConfig

② 设置所用到的中断的优先级别, 对每个中断调用函数 NVIC_Init()

使用IO口外部中断的一般步骤:

- ① 初始化IO口为输入
- ② 开启AFIO时钟
- ③ 设置IO口与中断线的映射关系
- ④ 初始化线上中断, 设置触发条件等
- ⑤ 配置中断分组(NVIC), 并使能中断
- ⑥ 编写中断服务函数

GPIO



EXTI



NVIC

GPIO_EXTIlineConfig(GPIO_PortSourceGPIOE,
GPIO_PinSource2);

PPP-ITConfig // 设置外设的某个(些)中断请求允许or禁止

PPP_GetITStatus // 获取外设PPP某中断状态

PPP_ClearITPendingBit // 清除外设PPP某挂起的中断标志

PPP_GetFlagStatus // 获取标志状态: 检测某外设标志是否置位 (状态标志位是否置位?)

PPP_ClearFlag // 挂起标志清除: 清除某外设挂起标志

(PPP指ADC、DMA、I2C、RCC、RTC、TIM、USART等具体外设)

例: 按键中断 — 根据按键状态控制LED灯亮灭

硬件连接: PC3 接 LED1

PE5 接 Key1

功能: PE5 配置为 EXTI 中断模式, Key1 按下时进入 EXTI 中断处理, 将 LED1 状态取反。

Key1 通过 PE5 配置为上拉输入, 无按键端口为高电平, 按键按下端口为低电平

PC3 控制共阳接法的 LED1, 端口输出1 则灯灭, 输出0 则灯亮

要在 stm32f10x-conf.h 中将 stm32f10x-gpio.h、stm32f10x-rcc.c、stm32f10x-exti.c、misc.c

这些头文件解除注释

主程序流程

```
int main(void)
```

```
{
```

```
    LED_GPIO_Config(); // LED 初始化, 配置LED用到的GPIO口
```

```
    LED1(ON); // LED1 初始状态为点亮状态
```

```
    EXTI-PE5-Config(); // 外部中断初始化(含KEY初始化), 配置PE5为线中断口, 并设置
```

```
    while(1) { // 循环等待中断
```

中断优先级

```
}
```


// PC3 控制 LED (共阳接法)

void LED_GPIO_Config(void)

```
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_APB2PeriphClockCmd (RCC_APB2Periph_GPIOC, ENABLE);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init (GPIOC, &GPIO_InitStructure);

    GPIO_SetBits (GPIOC, GPIO_Pin_3);
}
```

// PE5 接 Key1

void EXTI_PE5_Config(void)

```
{
    GPIO_InitTypeDef GPIO_InitStructure;
    EXTI_InitTypeDef EXTI_InitStructure;
    RCC_APB2PeriphClockCmd (RCC_APB2Periph_GPIOE | RCC_APB2Periph_AFIO, ENABLE);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU; // 上拉输入
    GPIO_Init (GPIOE, &GPIO_InitStructure);

    GPIO_EXTILineConfig (GPIO_PortSource_GPIOE, GPIO_PinSource5);
    EXTI_InitStructure.EXTI_Line = EXTI_Line5;
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling; // 下降沿中断
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_Init (&EXTI_InitStructure);

    NVIC_Configuration();
}
```


// NVIC 初始化配置

```
static void NVIC-Configuration(void)
```

```
{  
    NVIC_InitTypeDef NVIC_InitStructure;
```

```
    NVIC-PriorityGroupConfig(NVIC-PriorityGroup-1);
```

// 配置 PE[10:0] 为中断源

```
    NVIC_InitStructure.NVIC-IRQChannel = EXTI9-5-IRQn;
```

```
    NVIC_InitStructure.NVIC-IRQChannelPreemptionPriority = 0;
```

```
    NVIC_InitStructure.NVIC-IRQChannelSubPriority = 0;
```

```
    NVIC_InitStructure.NVIC-IRQChannelCmd = ENABLE;
```

```
    NVIC_Init(&NVIC_InitStructure);
```

```
}
```

// 在 stm32f10x-it.c 中编写中断服务程序

KEY1 (PE5) 的中断服务函数

```
void EXTI9-5-IRQHandler(void)
```

```
{
```

```
    if (EXTI-GetITStatus(EXTI-Line5) != RESET) // 判断是否产生了 EXTI-Line5 中断
```

```
{
```

// LED1 状态取反

```
    GPIO_WriteBit(GPIOC, GPIO-Pin-3,
```

```
        (BitAction) (1-GPIO_ReadOutputDataBit(GPIOC, GPIO-Pin-3)));
```

```
    EXTI-ClearITPendingBit(EXTI-Line5); // 清除中断标志位
```

```
}
```

```
}
```